

Lecture 2: Inside a Language Model

Allen Liu | CSCI-GA.3033-141

September 15, 2026

Last lecture, we treated a language model as a next-token predictor. We now look inside the model.

1 From tokens to vectors, and back

In *autoregressive generation*, we predict a next token, append it, and repeat. Label the vocabulary's m tokens by $1, \dots, m$. Given tokens $t_1, \dots, t_T$, the model computes one score per possible next token: the *logits*, $z_T \in \mathbb{R}^m$. Softmax turns them into probabilities:

$$p(t_{T+1} = j \mid t_1, \dots, t_T) = \frac{\exp(z_{T,j})}{\sum_{r=1}^m \exp(z_{T,r})}.$$

Embedding maps tokens to vectors; *unembedding* maps the network's final vectors to logits. All vectors are columns, and we omit biases.

An embedding assigns a learned d -dimensional vector to each token. Collect them as columns of $E \in \mathbb{R}^{d \times m}$. At position i ,

$$x_i = E_{t_i} \in \mathbb{R}^d,$$

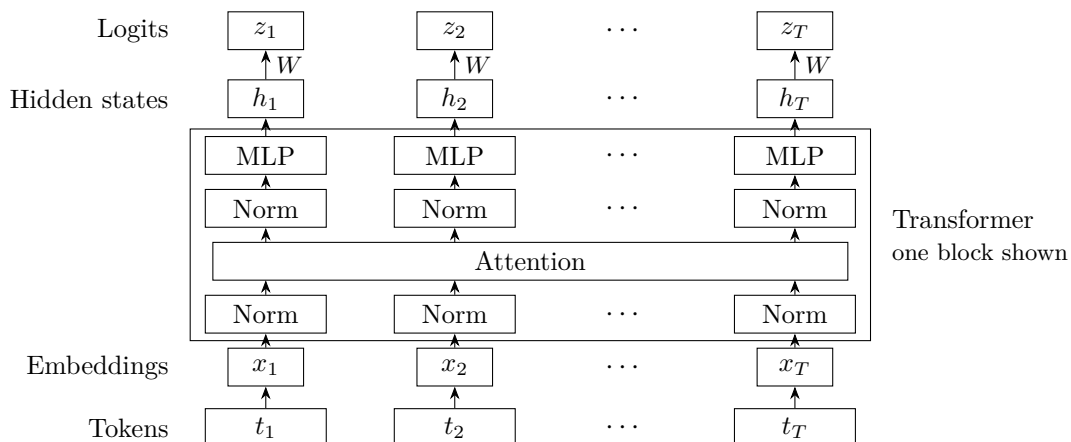
where E_{t_i} is the column for token t_i . Two occurrences of the same token start with the same embedding.

The transformer produces final *hidden states* $h_1, \dots, h_T \in \mathbb{R}^d$ that incorporate information from the sequence. A learned unembedding matrix $W \in \mathbb{R}^{m \times d}$ maps them to logits:

$$z_i = Wh_i.$$

The logits z_i predict the token at position $i + 1$. At the final position, softmax and sampling produce the next token t_{T+1} .

Transformer A transformer maps the input token embeddings to the hidden states at the last layer. A schematic is shown below (read from bottom to top).



A transformer combines three components: an MLP changes each position’s features, normalization controls vector scale, and attention reads other positions. We introduce them in that order. Within a transformer block, x_i denotes the current vector at position i ; h_i denotes the final vector.

2 A first building block: the MLP

A *multilayer perceptron* (MLP) changes one vector without reading other positions. It combines learned linear maps with nonlinear functions:

$$\text{MLP}(x) = W_2 \sigma(W_1 x).$$

W_1 maps the input to a vector usually of the same or a larger dimension. Each coordinate of $W_1 x$ measures a learned feature. The *activation* σ acts coordinatewise, and W_2 combines the resulting features into a d -dimensional vector.

For example, $\text{ReLU}(u) = \max(0, u)$ sets negative entries to zero, so the input determines which features contribute to the output.

The activation lets the MLP represent more than a linear map: composing linear maps alone gives $W_2(W_1 x) = (W_2 W_1)x$.

Within a block, the same MLP acts separately at every position. MLPs at different layers have different weights, but within a layer, the same MLP is applied at every position.

Modern MLPs often use smooth activations. One is *SiLU*, also called Swish:

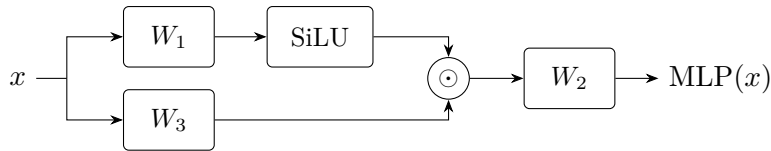
$$\text{SiLU}(u) = \frac{u}{1 + e^{-u}}.$$

It multiplies u by the *sigmoid* $1/(1 + e^{-u})$, a smooth weight between zero and one. SiLU approaches u for large positive inputs and zero for large negative inputs, with small negative outputs near zero. GELU is another smooth activation with a similar shape.

A *gate* lets one learned feature control another. LLaMA uses *SwiGLU*, which combines SiLU with a second linear branch:

$$\text{MLP}(x) = W_2(\text{SiLU}(W_1 x) \odot (W_3 x)).$$

Here $\odot$ means coordinatewise multiplication. The two branches have the same intermediate dimension: $W_3 x$ supplies features, and $\text{SiLU}(W_1 x)$ supplies multipliers that can suppress, amplify, or reverse their signs. Then W_2 maps the products back to dimension d .



3 Normalization: controlling vector scale

Repeated matrix multiplications can make vector magnitudes grow or shrink. Normalization controls this scale to help stabilize training. It acts separately on each position’s d coordinates.

We will use *RMSNorm*, a common choice in modern LLMs. It divides a vector by the root mean square of its coordinates, then applies learned coordinate scales:

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\|x\|_2^2/d + \varepsilon}}.$$

Here $\gamma \in \mathbb{R}^d$ is learned, and the small constant $\varepsilon > 0$ keeps the denominator positive.

Each normalization layer has its own γ , shared across positions; the denominator is computed from the current input. LayerNorm also subtracts the coordinate mean, then divides by a stabilized standard deviation and applies learned scales and shifts.

4 Self-attention: reading from other positions

The MLP and normalization act on each position separately. *Self-attention* reads other positions by taking a weighted average of their information, with weights determined by the input.

4.1 Queries, keys, and values

An attention *head* computes three vectors from each input x_i :

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i.$$

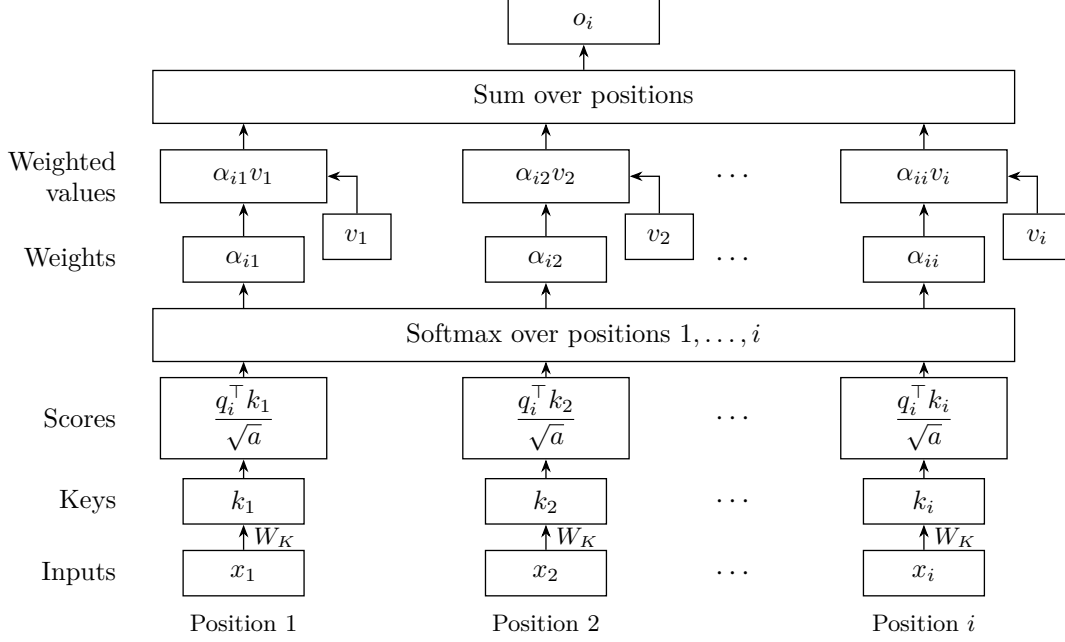
Each learned matrix maps $\mathbb{R}^d$ to $\mathbb{R}^a$, where a is the head dimension. The vectors have different jobs:

- The *query* q_i describes what position i is looking for.
- The *key* k_j is compared with the query to score position j .
- The *value* v_j is the information read from that position.

In *causal* attention, position i reads only positions $j \leq i$. Compare its query with each allowed key, apply softmax, and average the values:

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{a})}{\sum_{r=1}^i \exp(q_i^\top k_r / \sqrt{a})}, \quad o_i = \sum_{j=1}^i \alpha_{ij} v_j.$$

For each i , the weights α_{ij} are nonnegative and sum to one. Keys determine where to read: a larger dot product gives a position more weight. Values determine what information is returned.



One attention head at position i . The same query $q_i = W_Q x_i$ scores every key; each weight scales the value $v_j = W_V x_j$ from that position.

Scaling the dot product. The dot product adds a terms. If all coordinates of q_i and k_j are independent with mean zero and variance one, then

$$\text{Var}(q_i^\top k_j) = a, \quad \text{Var}(q_i^\top k_j / \sqrt{a}) = 1.$$

Dividing by $\sqrt{a}$ therefore keeps the initial scores at a comparable scale as the head dimension changes.

4.2 The matrix calculation and causal mask

While the above explains how to compute attention for one token, it is often useful to think of attention as a parallel operation, computed over all tokens in parallel. This is important especially in the prefill step (where you want to process a large fixed prefix). Of course, it is important to keep in mind the constraint that tokens should only attend to tokens before them.

To compute all positions together, let $X \in \mathbb{R}^{T \times d}$ have row i equal to $x_i^\top$. Then

$$Q = XW_Q^\top, \quad K = XW_K^\top, \quad V = XW_V^\top \in \mathbb{R}^{T \times a}.$$

The dot products form a $T \times T$ matrix. To exclude future positions, add a *mask* M before applying softmax:

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{a}} + M\right)V, \quad M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Softmax acts on each row, and $-\infty$ gives weight zero. Row i of O is $o_i^\top$. Position i may read itself because its output predicts the next position.

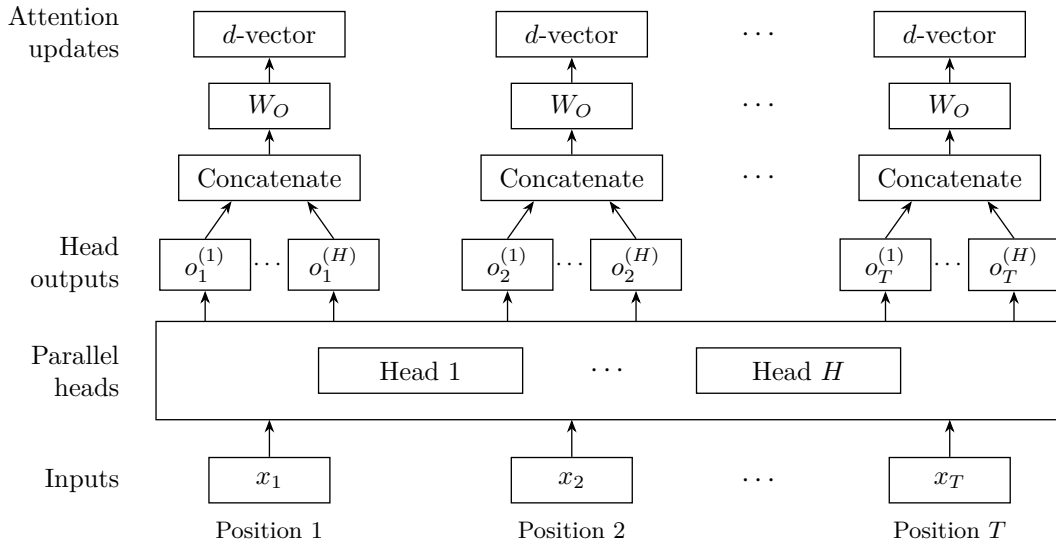
4.3 Multiple heads

Multi-head attention runs H heads so each position can read different kinds of information at once. Each head receives the full d -dimensional input and uses its own query, key, and value matrices to produce an a -dimensional output.

To combine the heads, concatenate their H output vectors into one vector of dimension Ha , then apply a learned linear map back to d . In matrix form,

$$\text{Attention}(X) = [O^{(1)} \mid \dots \mid O^{(H)}] W_O^\top \in \mathbb{R}^{T \times d}.$$

The brackets join the head outputs side by side into a $T \times (Ha)$ matrix. The learned map $W_O \in \mathbb{R}^{d \times Ha}$ appears transposed because the vectors are stored as rows. A common choice is $Ha = d$.



Every head receives the same input sequence. At each position, concatenate the head outputs and apply the shared output map W_O .

5 Position information and RoPE

Attention as written above is invariant to the order of the tokens before the current token. This is not ideal as we would like our model to be able to encode positional information. We describe the mechanism for this below.

Rotary position embeddings (RoPE) rotate queries and keys at each layer. In two dimensions, a rotation through angle θ is

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}.$$

Choose an angle step ω . At position i , rotate by $i\omega$. The new query–key dot product is

$$(R(i\omega)q_i)^\top (R(j\omega)k_j) = q_i^\top R((j-i)\omega)k_j.$$

This follows from $R(\theta)^\top = R(-\theta)$ and the rule that composing rotations adds their angles. The position term depends on the *relative offset* $j-i$, and rotations preserve vector lengths.

For larger vectors, rotate each coordinate pair with a different angle step, giving several scales for representing distance. Attention uses the rotated queries and keys in its dot products; values stay unchanged. LLaMA and Qwen3 use RoPE.

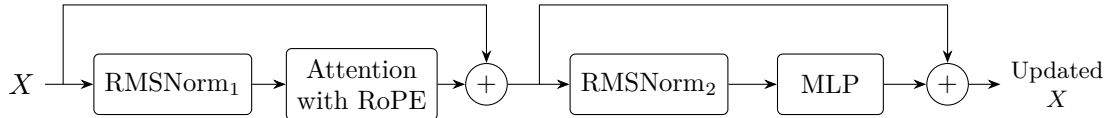
6 Assembling a transformer

We now combine the three components. The matrix $X \in \mathbb{R}^{T \times d}$ stores the current vector at each position: row i is $x_i^\top$. Initially these rows contain the token embeddings; each block updates X while keeping its shape $T \times d$.

In a *pre-norm* transformer block, normalize before attention and before the MLP, then add each operation's output to its input:

$$\begin{aligned} X &\leftarrow X + \text{Attention}(\text{RMSNorm}_1(X)), \\ X &\leftarrow X + \text{MLP}(\text{RMSNorm}_2(X)). \end{aligned}$$

These updates are sequential. The additions, called *residual connections*, preserve the incoming vector and add a learned update. The MLP and RMSNorm act separately on the vectors stored in each row; the two normalization sites have separate learned scales.



After L blocks with separate weights, apply a final RMSNorm to each row of X . Row i is then $h_i^\top$, where $h_i \in \mathbb{R}^d$ is the final hidden state at position i . It incorporates information from tokens $t_1, \dots, t_i$; $z_i = Wh_i$, followed by softmax, predicts the token at position $i + 1$. There are $2L + 1$ normalization sites in total. With RoPE, position information enters through attention.

Parameter counts follow from the matrix dimensions. For ordinary attention with $Ha = d$, the query, key, value, and output maps contain $4d^2$ weights. The simple MLP contains $2d$ times its intermediate dimension; the gated version contains $3d$ times that dimension. The embedding and output matrices together contain $2md$ weights, or md if $W = E^\top$. These are the weights in the main linear maps.

6.1 A sense of scale

A *dense* transformer uses all MLP weights for every token in a block. Qwen3-8B is one example:

Vector width d	4096
Blocks L	36
Attention heads H / head dimension a	32 / 128
MLP intermediate dimension	12288
Vocabulary size m	151936
Native context length	32768

7 Modern modifications

Modern LLMs now have several modifications on top of the vanilla transformer to improve their performance and efficiency.

7.1 Attention variants

First we discuss variants of attention. To understand the purpose and power of attention, it will be instructive to think about older architectures from before attention was introduced.

7.1.1 Thought experiment: a recurrent language model

A *recurrent model* reads one token at a time, updating a hidden state that contains all of the model’s information about the prefix so far:

$$s_i = f(s_{i-1}, x_i).$$

From the state s_i , the model predicts the next token, and the learned update f is reused at every step. Suppose s_i has a fixed number of coordinates d . This type of recurrent model has a key limitation in terms of long-range memory. We can see this clearly through the task of copying.

A counting bound for copying. The task of copying is: given T tokens and then a special token that instructs the model to copy, to copy exactly those T tokens i.e. the next T tokens the model outputs should match the prefix the model is given.

We now show why recurrent architectures are limited in their ability to copy. Suppose the model reads T tokens, then receives a fixed instruction to reproduce them in order. Its entire memory at this point has at most $B = O(d)$ bits (assuming bounded floating point precision), and the distribution of the model’s outputs over the next T tokens is a deterministic function of that memory.

There are m^T possible strings and at most 2^B memory states. However, each state needs to produce a different answer with high probability, so at most one string per state is copied correctly. For a uniformly random input,

$$\Pr[\text{exact copy}] \leq \min\left(1, \frac{2^B}{m^T}\right).$$

Copying every string correctly therefore requires

$$B \geq T \log_2 m.$$

Regardless of f , copying every string requires memory linear in T . At fixed precision, the number of state coordinates must therefore grow with T .

7.1.2 Transformer state and the KV cache

During generation, a transformer also processes tokens one at a time. What state does it carry between steps?

The prefix is stored in the *KV cache*: past keys and values at every layer. With fixed weights and positions, causal attention leaves past vectors unchanged when a token is appended, so their keys and values can be reused.

At each layer, compute the new token’s query, key, and value; append its key and value to the cache; and let its query read the expanded cache. The final vector predicts the next token, and the expanded cache is the new recurrent state. With RoPE, the cached keys already include their rotations.

The cache grows with the prefix, providing the memory scaling required by the copying bound. Attention can use it to copy by retaining token identities in values and retrieving successive input positions, as in the earlier lookup example.

7.1.3 Computation and memory costs

The KV cache scaling with T is necessary for giving attention its power, but also a curse for computational reasons. Storing and reading this growing state makes long contexts expensive. For one sequence of length T , each of the H heads in each of the L layers stores T keys and T values of dimension a . Thus

$$\text{cache size} = 2LTHa \times \text{bytes per number.}$$

Longer contexts leave room for fewer simultaneous sequences in device memory. Each new token also reads more keys and values, increasing computation and data movement.

With $Ha = d$, comparing queries with keys and averaging values costs $O(Td)$ operations per layer for one new token. For a whole prefix, these operations cost $O(T^2d)$ per layer, since there are quadratically many position pairs.

These costs, specifically reducing the KV cache, motivate a variety of recent changes to attention.

7.1.4 Sharing keys and values

Grouped-query attention (GQA) divides the H query heads into G groups. Heads in a group share keys and values but keep separate queries and attention weights. Replacing H by G in the cache formula reduces storage by a factor of H/G .

Qwen3-8B uses $G = 8$ for 32 query heads. Sharing one set across all heads ($G = 1$) is called *multi-query attention*.

7.1.5 Sliding-window attention

Sliding-window attention lets each position read itself and the previous $w - 1$ positions for a window w that is much smaller than the context length. With $Ha = d$, comparing queries with keys and averaging values costs $O(wd)$ per new token per layer. Each head stores at most w key/value pairs. Information can travel farther by passing through several layers. Stacking L windowed layers lets a position receive information from up to $L(w - 1)$ positions earlier.

7.1.6 Linear attention

A different approach compresses the past into running sums. For a simple version, make queries and keys positive coordinatewise by applying a positive activation after their linear maps. Continue to write these vectors as q_i and k_i . Replace softmax’s exponential weights with dot products:

$$o_i = \frac{\sum_{j=1}^i (q_i^\top k_j) v_j}{\sum_{j=1}^i q_i^\top k_j}.$$

Positivity ensures a nonzero denominator and nonnegative weights. Collect the past into two sums:

$$S_i = \sum_{j=1}^i v_j k_j^\top \in \mathbb{R}^{a \times a}, \quad u_i = \sum_{j=1}^i k_j \in \mathbb{R}^a.$$

Rearranging the products gives the output and recurrent updates:

$$o_i = \frac{S_i q_i}{u_i^\top q_i}, \quad S_i = S_{i-1} + v_i k_i^\top, \quad u_i = u_{i-1} + k_i.$$

Starting from $S_0 = 0$ and $u_0 = 0$, the state uses $a^2 + a$ numbers per head. Each update and read costs $O(a^2)$ operations. For fixed a , total work is linear in sequence length, hence the name *linear attention*. At fixed precision, the copying bound applies to this fixed-size state as well.

7.1.7 Hybrid models

The above modifications are less expensive than full attention but used alone, would sacrifice significantly in terms of performance. Thus, modern architectures (e.g. Kimi K3, Deepseek v4 etc.) are often hybrid architectures that use a combination of full attention layers and cheaper layers in some ratio. This way, models can often use, say, 1/4 the number of full attention layers (thus reducing KV cache by essentially a factor of 4), with minimal cost to model performance.

7.2 Mixture of experts (MoE)

A recent trend is that model training is an upfront one-time cost while inference is repeated many times when serving the model. We can afford more training computation if it improves quality while keeping each use cheap.

Imagine training several models that specialize in different inputs, with a small router selecting one per request. We train and store all the models, but each request evaluates only the router and one model.

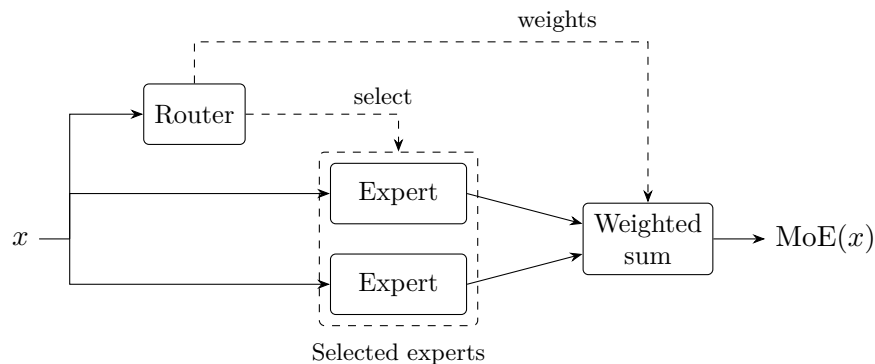
This thought experiment brings us to MoE. A *mixture-of-experts* (MoE) block brings this choice inside the model: it replaces the MLP with several MLPs, called *experts*, each with its own weights. A learned *router* chooses which experts to use for each token. A simple router computes one score per expert:

$$g = W_R x,$$

where W_R has one row per expert and d columns. Let I contain the indices of a few experts with the largest scores. Apply softmax over those scores and combine the selected outputs:

$$\text{MoE}(x) = \sum_{j \in I} \frac{e^{g_j}}{\sum_{r \in I} e^{g_r}} \text{MLP}_j(x).$$

Each selected expert MLP_j maps the same input x to a d -dimensional output; their weighted sum replaces the MLP output. The selection I depends on x , while router and expert weights are shared across positions.



Adding experts increases the total parameter count while the number evaluated per token can stay fixed.

Many MoE models also include *shared experts*: MLPs evaluated for every token, regardless of the router's selection. Their outputs are added to the weighted sum from the routed experts. The aim is to learn features useful across many tokens, leaving the routed experts more capacity to specialize.